INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	1/11/2026
Enlace de Git	https://github.com/B3kk4r/Portafolio-de-Base-de-Datos.git
Nombres y apellidos del estudiantes	Isaac Alejandro Vire Poma

1. TITULO

Diseño e Implementación de la Base de Datos para la tienda de videojuegos "PixelVerse Store"

2. CONTEXTO DEL PROBLEMA

- **¿Cuál es la situación o entorno donde se presenta el problema?**

La problemática gira en un entorno operativo y administrativo de la tienda de videojuegos “PixelVerse Store”. Esta tienda ofrece un gran catálogo de videojuegos que varían según la plataforma y la desarrolladora detrás del juego, atienden constantemente a una gran cantidad de clientes que realizan compras individuales o múltiples.

- **¿Qué dificultades o necesidades existen actualmente sin una base de datos?**

Hoy en día, el registro y almacenamiento de transacciones de compra y venta se realizan de forma manual y lenta, lo que provoca pérdida de ingresos. De la misma manera, no se cuenta con un registro centralizado del stock disponible, lo que impide saber qué títulos están agotados o a qué plataforma pertenecen. Por último, no se puede guardar un historial de los clientes ni del valor al que se vendió un producto en el pasado.

- **¿A quiénes afecta el problema? (usuarios, institución, organización)**

Esto afecta de manera inmediata a los propietarios del negocio que no pueden gestionar el negocio correctamente, mantener un orden contable y planificar el inventario a futuro tiendo en cuenta los resultados presentes. De igual manera afecta a los clientes, que tienen que esperar más tiempo del necesario para que su compra se procesada, lo que genera malestar y evita que prefieran la marca sobre otras tiendas.

- **¿Qué datos se necesitan gestionar y por qué es importante hacerlo correctamente?**

Para solucionar esta problemática, el sistema necesita gestionar de forma correcta cinco flujos de datos principales: la información personal de los **clientes**, el catálogo detallado de **videojuegos**, con precios y stock, la clasificación por **plataformas**, el origen de los estudios **desarrolladores**, y el registro de **ventas** junto con su respectivo **detalle** de la línea de compra.
Gestionar estos datos correctamente es vital para garantizar la consistencia de la información, asegurar la transparencia financiera de la tienda y permitir análisis comerciales futuros, como identificar los videojuegos o plataformas y desarrolladoras más rentables para el negocio.

3. OBJETIVOS

3.1. Objetivo General

Desarrollar un sistema de base de datos para la gestión comercial de la tienda “PixelVerse Store” mediante el diseño de un modelo entidad-relación normalizado y su posterior implementación en lenguaje SQL, para centralizar el registro de clientes, catálogo de videojuegos y ventas, y facilitar el control del inventario y la obtención de información comercial.

3.2. Objetivos Específicos

1. Construir el modelo entidad-relación y el esquema relacional de la base de datos de “PixelVerse Store” mediante la aplicación de reglas y definiciones de atributos correspondientes a las entidades, con el fin de garantizar la integridad de la información a utilizar y evitar problemas futuros.

2. Desarrollar el script de bases de datos utilizando el lenguaje estructurado SQL para la creación de las tablas, restricciones, llaves foráneas y la posterior implementación de datos, para analizar y garantizar un procesamiento ágil y seguro de las ventas realizadas.

3. Formular consultas en álgebra relacional y traducirlas a SQL, aplicando operadores como selección, proyección, producto cartesiano, join, diferencia, unión y división, para extraer información útil sobre clientes, catálogo y ventas de la tienda.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

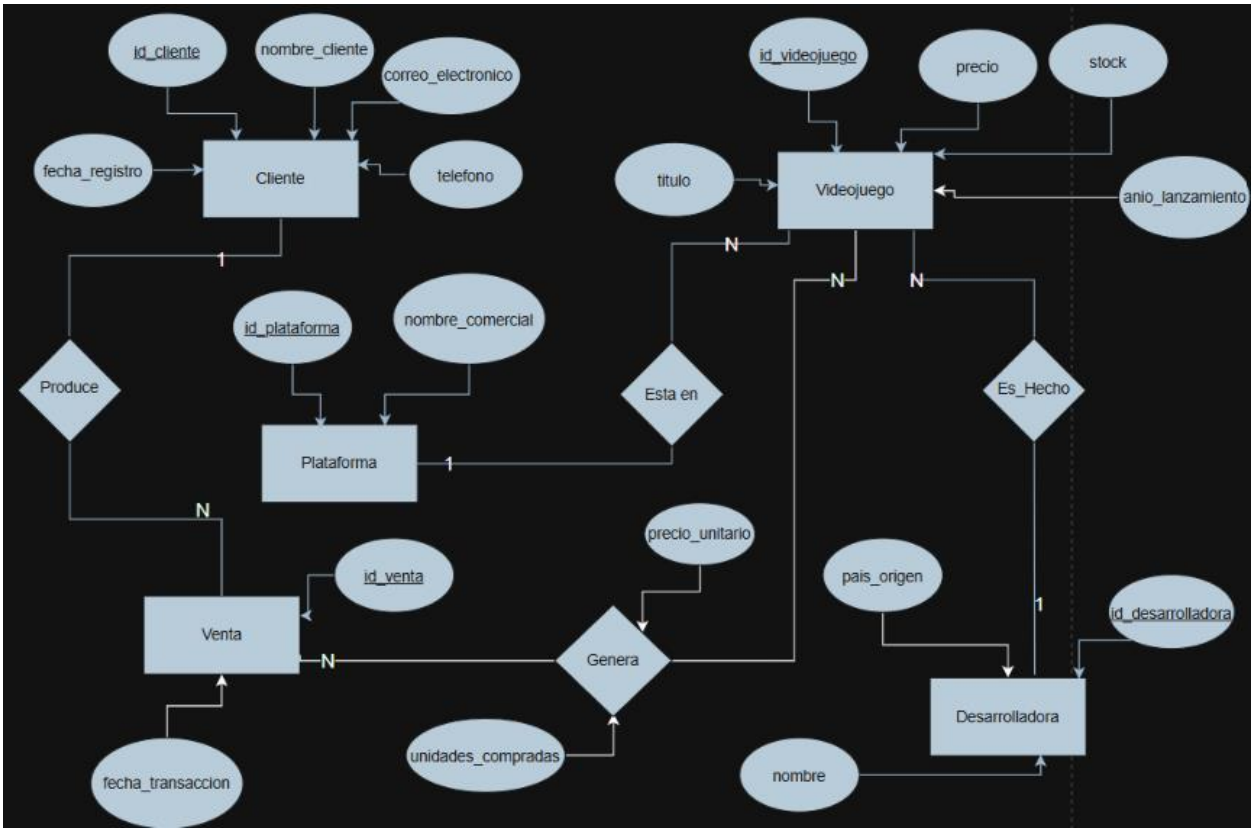


Figura 1. Diagrama Entidad-Relación del sistema para la tienda de videojuegos

4.2. Modelo Relacional

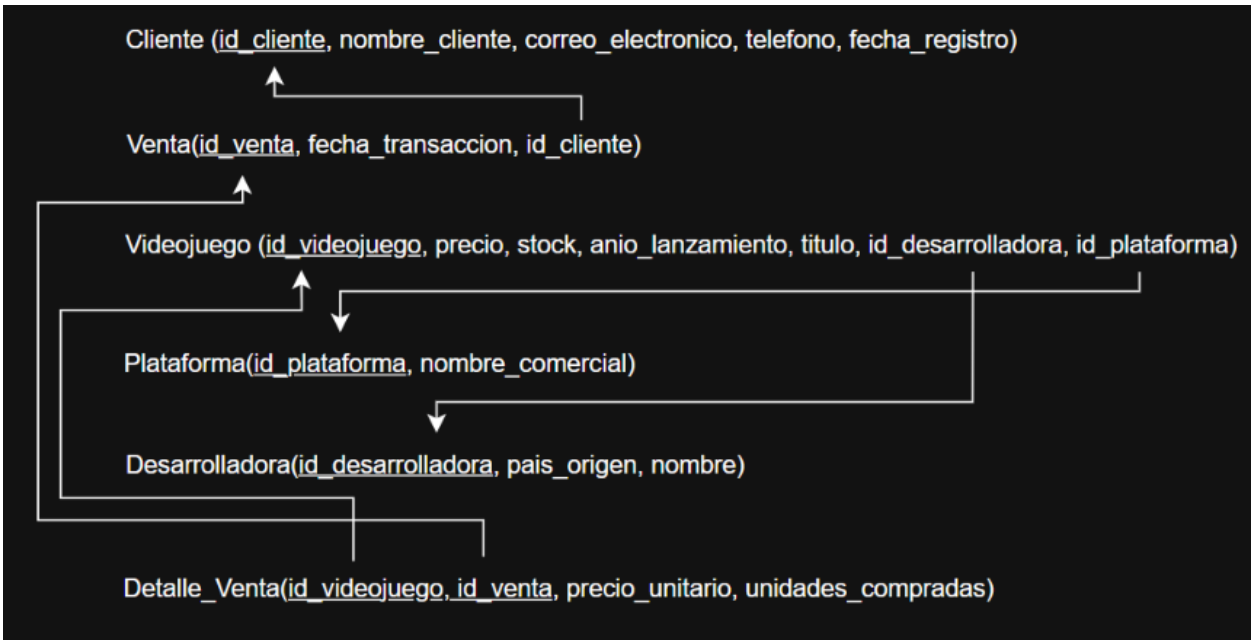


Figura 2. Diseño Relacional del sistema para la tienda de videojuegos.

4.3. Decisión de diseño

La construcción de la base de datos se realizo con lo siguiente en mente.

- **Un videojuego pertenece a una sola plataforma.** Cada registro de Videojuego representa un título en una plataforma específica, con su propio precio y stock. Si un mismo juego se vende en dos plataformas, se registra dos veces.
- **Cada videojuego tiene una única desarrolladora**, y una desarrolladora puede tener varios videojuegos.
- **Una venta pertenece siempre a un cliente registrado.** No se contemplan ventas anónimas, por lo que id_cliente es obligatorio en Venta.
- **La relación entre Venta y Videojuego se resuelve con Detalle_Venta**, cuya llave primaria compuesta evita repetir un mismo videojuego dentro de una venta, dado que la cantidad se registra en unidades_compradas.
- **El precio de venta se guarda en Detalle_Venta** (precio_unitario) y no solo en Videojuego, para conservar el valor al que realmente se vendió el producto, aunque el precio del catálogo cambie después.
- **El stock se almacena como un atributo del videojuego.** La base de datos permite consultarlo, pero su descuento al registrar una venta no es automático, se tendría que incluir un trigger que se ejecute, por ejemplo, con cada nueva venta realizada.
- El modelo cubre la venta a clientes. No incluye proveedores, compras de reposición, empleados ni métodos de pago.

La mayoría de estas decisiones fueron tomadas para generar una base de datos no muy compleja ni tardada de diseñar y analizar.

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

```
create table Cliente (
	id_cliente int primary key AUTO_INCREMENT,
	nombre_cliente varchar (30) NOT NULL,
	correo_electronico varchar (60) NOT NULL UNIQUE,
	telefono varchar (10),
	fecha_registro DATE NOT NULL DEFAULT (CURRENT_DATE)
);

create table Venta (
	id_venta int primary key AUTO_INCREMENT,
	fecha_transaccion DATE NOT NULL,
	id_cliente int NOT NULL,
	Foreign Key (id_cliente) references Cliente (id_cliente)
);

create table Desarrolladora (
	id_desarrolladora int primary key AUTO_INCREMENT,
	pais_origen varchar (50) NOT NULL,
	nombre varchar (50) NOT NULL
);

create table Plataforma (
	id_plataforma int primary key AUTO_INCREMENT,
	nombre_comercial varchar (50) NOT NULL
);

create table Videojuego (
	id_videojuego int primary key AUTO_INCREMENT,
	precio DECIMAL(10,2) NOT NULL,
	stock int NOT NULL,
	anio_lanzamiento int NOT NULL,
	titulo varchar (50) NOT NULL,
	id_desarrolladora int NOT NULL,
	id_plataforma int NOT NULL,
	Foreign Key (id_desarrolladora) references Desarrolladora (id_desarrolladora),
	Foreign Key (id_plataforma) references Plataforma (id_plataforma)
```

```
);

create table Detalle_Venta (
    id_videojuego int NOT NULL,
    id_venta int NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    unidades_compradas int NOT NULL CHECK (unidades_compradas > 0),
    PRIMARY KEY (id_videojuego, id_venta),
    Foreign Key (id_videojuego) references Videojuego (id_videojuego),
    Foreign Key (id_venta) references Venta (id_venta)
);
```

5.2. Inserción de Datos (DML)

```
INSERT INTO Cliente (nombre_cliente, correo_electronico, telefono, fecha_registro) VALUES
('Carlos Vire', 'carlos.vire@email.com', '555123456', '2025-01-10'),
('Ana Rodríguez', 'ana.rod@email.com', '555987654', '2024-02-15'),
('Luis Poma', 'luis.poma@email.com', NULL, '2023-03-01'),
('Kira Martínez', 'kira.m@email.com', '556556789', '2024-03-05'),
('Lucio Torres', 'lucio.t@email.com', '575689123', '2023-04-20');

INSERT INTO Desarrolladora (pais_origen, nombre) VALUES
('Japón', 'Nintendo'),
('Estados Unidos', 'Santa Monica Studio'),
('Reino Unido', 'Rockstar North'),
('Polonia', 'CD Projekt Red'),
('Japón', 'FromSoftware'),
('Australia', 'Team Cherry');

INSERT INTO Plataforma (nombre_comercial)
VALUES
('Nintendo Switch'),
('PlayStation 5'),
('PC');

INSERT INTO Venta (fecha_transaccion, id_cliente)
VALUES
('2025-02-01', 1),
('2025-03-12', 1),
('2024-05-20', 2),
('2025-04-05', 1),
```

```
('2025-06-18', 1);


INSERT INTO Videojuego (precio, stock, anio_lanzamiento, titulo, id_desarrolladora, id_plataforma) VALUES
(59.99, 15, 2023, 'The Legend of Zelda: ToTK', 1, 1),
(69.99, 8, 2022, 'God of War Ragnarok', 2, 2),
(29.99, 50, 2013, 'Grand Theft Auto V', 3, 3),
(49.99, 22, 2020, 'Cyberpunk 2077', 4, 3),
(69.99, 0, 2022, 'Elden Ring', 5, 2),
(14.99, 30, 2017, 'Hollow Knight', 6, 1);


INSERT INTO Detalle_Venta(id_videojuego, id_venta, precio_unitario, unidades_compradas) VALUES
(1,1,59.99,1),(2,1,69.99,1),(4,2,49.99,2),(1,3,59.99,1),(6,3,14.99,1),(3,4,29.99,1),(6,4,14.99,1),(5,5,69.99,3);
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

Consulta 1-(operadores: selección y proyección)

El título y el precio de todos los videojuegos que tengan un precio de venta superior a $50.00.

Álgebra Relacional:

Π titulo, precio (σ precio > 50.00 (Videojuego))

SQL:

```
SELECT titulo, precio FROM Videojuego WHERE precio > 50.00;
```

Resultado esperado: Al ejecutar esta consulta se obtendrá una relación con el titulo y precio de los videojuegos con precio de venta superior a 50 dólares. Tomando los datos ingresados en la sección 5.2, saldría: Zelda, God of War Ragnarok y Elden Ring.

Consulta 2-(operadores: proyección, producto cartesiano y selección)

El nombre del cliente y la fecha de transacción de todas las ventas que se han realizado en la tienda.

Álgebra Relacional:

Π Cliente.nombre_cliente, Venta.fecha_transaccion (*σ* Cliente.id_cliente = Venta.id_cliente (Cliente x Venta))

SQL:

```
SELECT Cliente.nombre_cliente, Venta.fecha_transaccion FROM Cliente, Venta WHERE
Cliente.id_cliente = Venta.id_cliente;
```

Resultado esperado: Al ejecutar esta consulta se obtendrá una relación con el nombre del cliente y la fecha de transacción de todas la ventas efectuadas. Tomando los datos ingresados en la sección 5.2, saldría: Carlos Vire con 4 ventas y Ana Rodríguez con 1.

Consulta 3-(operadores: diferencia, proyección y Join)

Obtener el identificador único y el nombre de todos los clientes registrados en el sistema que **no** han realizado ninguna compra, ninguna venta. en la tienda hasta el momento.

Álgebra Relacional:

Clientes_Sin_Compra = Π id_cliente (Cliente) – Π id_cliente(Venta)
Π id_cliente, nombre_cliente (Cliente ⋈ Clientes_Sin_Compra)

SQL:
SELECT Cliente.id_cliente, Cliente.nombre_cliente FROM Cliente LEFT JOIN Venta ON Cliente.id_cliente = Venta.id_cliente WHERE Venta.id_cliente IS NULL;

Resultado esperado: Al ejecutar esta consulta se mostrará una relación con el identificador único y el nombre de los clientes que estén en el sistema pero que no hayan realizado una compra. Tomando los datos ingresados en la sección 5.2, saldría: Luis Poma, Kira Martínez y Lucio Torres.

Consulta 4-(operadores: unión, selección, proyección y Join)
Los nombres de todos los estudios desarrolladores que tienen su origen en "Japón" **o** que han creado videojuegos para la plataforma "Nintendo Switch".

Álgebra Relacional:

Nintendo_Switch = Π Desarrolladora.nombre (σ Plataforma.nombre_comercial = “Nintendo Switch” (Plataforma ⋈ Videojuego ⋈ Desarrolladora))

Estudios_Japon = Π nombre (σ pais_origen = “Japón”(Desarrolladora))

Nintendo_Switch U Estudios_Japon

SQL:
SELECT Desarrolladora.nombre
FROM Plataforma
INNER JOIN Videojuego ON Plataforma.id_plataforma = Videojuego.id_plataforma
INNER JOIN Desarrolladora ON Videojuego.id_desarrolladora = Desarrolladora.id_desarrolladora
WHERE Plataforma.nombre_comercial = 'Nintendo Switch'

UNION

SELECT nombre
FROM Desarrolladora
WHERE pais_origen = 'Japón';

Resultado esperado: Al ejecutar esta consulta se verá una relación que incluya tanto a las desarrolladoras que tienen origen en Japon como a las que han creado juegos para la Nintendo Switch. Tomando los datos ingresados en la sección 5.2, saldría: Nintendo, FromSoftware y Team Cherry.

Consulta 5-(operadores: división, proyección y Join)
Los identificadores únicos y nombres de los clientes que han comprado todos los videojuegos disponibles en el catálogo de la tienda.

Álgebra Relacional:

Historial = Π id_cliente, id_videojuego (Venta ⋈ Detalle_Venta)

Catalogo = Π id_videojuego (Videojuego)

Cliente_id = Historial ÷ Catalogo

Π id_cliente, nombre_cliente (Cliente ⋈ Cliente_id)

SQL:

```
SELECT id_cliente, nombre_cliente
FROM Cliente
WHERE NOT EXISTS (
   SELECT id_videojuego
   FROM Videojuego
   WHERE NOT EXISTS (
      SELECT 1
      FROM Venta
      INNER JOIN Detalle_Venta ON Venta.id_venta = Detalle_Venta.id_venta
      WHERE Venta.id_cliente = Cliente.id_cliente
      AND Detalle_Venta.id_videojuego = Videojuego.id_videojuego
   )
);
```

Resultado esperado: Al ejecutar esta consulta lo que se obtendrá será solamente los nombres e identificadores de los clientes que hayan comprado cada uno de los videojuegos ofrecidos por la tienda. Tomando los datos ingresados en la sección 5.2, saldría: Carlos Vire

CONCLUSIONES

- Conclusión 1: La realización del modelo entidad-relacion y el modelo relacional trajo consigo un reto más que todo en el apartado de la coherencia entre el contenido que ambos se supone que tienen. Es fácil confundirse y omitir información que pueda afectar gravemente la creación de un base de datos en condiciones. De cualquier manera, al ser un proceso repetitivo, se va obteniendo experiencia poco a poco que evita que estos problemas surjan constantemente.
- Conclusión 2: La escritura de código ayudo a comprender la manera en la que una base de datos relacional esta hecha al permitir, de cierta forma, ver sus entrañas y la manera tan cuidadosa y exacta estaba todo declarado, al mínimo fallo un tabla no se creaba o si lo hacia presentaba problemas para guardar fielmente lo que se supone que tenía que contener. Los aspectos más relevantes fueron el orden que se tenia que considerar para que una línea de código estuviera bien escrita y las palabras reservadas que podían cambiar por completo el dominio o comportamiento de una columna.
- Conclusión 3: El uso del algebra relacional permitió ver la manera en la que se tenían que interpretar las cosas a la hora de sacar información, pues al poder entenderlo de forma más apegada a nuestro idioma, se puede entender más rápidamente lo que el lenguaje SQL está haciendo. Las consultas más útiles son aquellas que seleccionan o proyecta la información requerida, junto al producto cartesiano y el join, que facilitan la navegabilidad entre distintas tablas.

BIBLIOGRAFÍA

- C. J. Sánchez Aldana y F. I. Mosquera Motta, Modelamiento de base de datos: Metodología práctica y aplicada. Bogotá, Colombia: Universidad Piloto de Colombia, 2020.

- Oracle Corporation, "Preface and Legal Notices," en *MySQL 9.7 Reference Manual*, 2026. [En línea]. Disponible en: https://dev.mysql.com/doc/refman/9.7/en/preface.html